Advanced Labs Report

Name: Andrea Trevanna Team: D

Student ID: 202318030

System Optimization

Objective: Optimization is the quest for strategies that make a given criterion as effective as possible. We learned several optimization algorithms: least squares method, steepest descent and neural networks, and combinatorial optimization such as genetic algorithm. In our final week, we then designed a free themed optimization project. Our team made the project apartment recommendation system for students around University of Tsukuba based on fitness and genetic algorithm.

Code are in :

https://drive.google.com/drive/folders/1AcDvvq0JPd0WEAXDwukgl1AEiG7NNOwF?usp=sharing

Attached ipnyb files for Task 8 and 9 contest (zip file)

For Free Project:

optimisation.ipynb - Colab

PPT is in Zip file

Optimization problem

An optimization problem aims to find the maximum or minimum value of a function f. Hereafter, this function will be referred to as the objective function f. In an optimization problem, the objective function f depends on several variables (which may be real-valued or integer-valued): x1,x2,...,xn. These variables are called decision variables. The solution method for an optimization problem refers to a method for finding the optimal values of the decision variables x1,x2,...,xn that maximize or minimize the objective function f.

Week 1 Least Squares Method for Data Fitting

Optimization problem:

An optimization problem aims to find the maximum or minimum value of a function f , that is, to find where the function attains its maximum or minimum value.Hereafter, this function will be referred to as the objective function f. In an optimization problem, the objective function f depends on several variables (which may be real-valued or integer-valued): x1,x2,...,xn (1) These variables are called decision variables. The solution method for an optimization problem refers to a method for finding the optimal values of the decision variables x1,x2,...,xn that maximize or minimize the objective function f.

For each programming exercise problem, we did in Google Colab and mounted dataset (CSV file format) to Google Colab from Google Drive using from google.colab import drive

Week 1: Least Squares Method

The method of least squares is a method of fitting a line (or curve) so as to minimize the sum of squared residuals between each data point and the line (or curve), thereby achieving the best possible approximation. Equivalently, it solves an optimization problem that seeks the line (or curve) that minimizes the sum of squared residuals.

In the method of least squares, we minimize the following objective function f.

$$f(a,b) = \frac{1}{2}\sum_{i=1}^{N}(y_i - (ax_i + b))^2 \quad (1)$$

The objective function f computes the squared difference between the fitted line and each data point, and then sums these values over all data points.

the objective function (Eq.4) is a function of the decision variables a and b. We apply the analytical method introduced in the section on the steepest descent method for finding the optimal value of an objective function in order to obtain the minimum of Eq.1

To obtain the values of a and b that minimize the objective function f.

$$\frac{\partial f}{\partial a} = a\sum_{i=1}^{N} x_i^2 + b\sum_{i=1}^{N} x_i - \sum_{i=1}^{N} x_i y_i = 0,$$
$$\frac{\partial f}{\partial b} = a\sum_{i=1}^{N} x_i + b\sum_{i=1}^{N} 1 - \sum_{i=1}^{N} y_i = 0,$$

Since a and b appear in a well-organized form, we can rewrite the above system as the following simultaneous linear equations:

$$\begin{pmatrix} \sum_{i=1}^{N} x_i^2 & \sum_{i=1}^{N} x_i \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^{N} x_i y_i \\ \sum_{i=1}^{N} y_i \end{pmatrix}$$

Solve the normal equations to obtain and and b.

Expressing this system using matrix and vector notation, we have

$$\boldsymbol{\Sigma v} = \boldsymbol{\sigma}$$

$$f(a,b) = \frac{1}{2}\sum_{i=1}^{N}\left(y_i - (ax_i + b)\right)^2 ,$$

In week 1, we reviewed the preliminaries including numpy and pandas. We then did some exercised regarding least squares method.

Exercise 1:

Conduction of a growth survey of a certain plant species on Mountains A through N in Japan. For each mountain, data were collected on the plant's radius xx (cm) and weight yy (g). Our team used Dataset of Mountain A. Each dataset is in CSV format. The first column contains the plant radius (radius), and the second column contains the plant weight (weight). We used the method of least squares to fit a straight-line y=ax+b to this dataset and obtain vales of a and b.

Exercise 1-1

Write the matrix calculations for the least squares method to find the parameters of the line of best fit. Also, calculate the sum of squared residuals at this time.

We wrote the code based on the matrix equation to solve the values a and b in the least squares.

$$\begin{pmatrix} \sum_{i=1}^{N} x_i^2 & \sum_{i=1}^{N} x_i \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} 1 \end{pmatrix} \begin{pmatrix} a \\ b \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^{N} x_i y_i \\ \sum_{i=1}^{N} y_i \end{pmatrix}$$

Expressing this system using matrix and vector notation,

$$\boldsymbol{\Sigma v} = \boldsymbol{\sigma}$$

Where

Where the matrix Σ and the vectors v and σ are defined as

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sum_{i=1}^{N} x_i^2 & \sum_{i=1}^{N} x_i \\ \sum_{i=1}^{N} x_i & \sum_{i=1}^{N} 1 \end{pmatrix}, \boldsymbol{v} = \begin{pmatrix} a \\ b \end{pmatrix}, \boldsymbol{\sigma} = \begin{pmatrix} \sum_{i=1}^{N} x_i y_i \\ \sum_{i=1}^{N} y_i \end{pmatrix}$$

To calculate a and b, compute the inverse matrix Σ−1and multiply both sides from the left to obtain

$$v = \Sigma^{-1}\sigma$$

Using the code in Figure 1 , we obtain a= 0.9625794860332839 and b= 1.1146519047176304

Thus, y=ax+b= 0.9625794860332839 x+ 1.1146519047176304

```
# \Sigmaと\sigmaの計算(numpyの行列の作り方，pandasによる要素ごとカラムごとの計算に注意)
# Calculation of \Sigma and \sigma (Pay attention to how to create matrices in numpy
# and how to perform element-wise/column-wise calculations in pandas)


# vの計算
# v = \Sigma^{-1}\sigmaを記述(逆行列の計算はnumpyで用意されているのを使うと良い)
# Calculation of v
# Describe v = \Sigma^{-1}\sigma (It is recommended to use the inverse matrix
# calculation provided by numpy)
x=data['radius']
y=data['weight']
N=len(x)

import numpy as np
Summation = np.array([
    [np.sum(x**2), np.sum(x)],
    [np.sum(x), N]])
sigma = np.array([
    np.sum(x * y),
    np.sum(y)
])
print("Summation(Σ):")
print(Summation)
print("\nSigma vector (σ):")
print(sigma)
a,b= np.linalg.inv(Summation) @ sigma
print("a=",a)
print("b=",b)
print("y=",a,"x+",b)
```

```
Summation(Σ):
[[3471.3212  216.54  ]
 [ 216.54     20.    ]]

Sigma vector (σ):
[3582.7893  230.73  ]
a= 0.9625794860332839
b= 1.1146519047176304
y= 0.9625794860332839 x+ 1.1146519047176304
```

Figure 1 Exercise 1-(1)

Exercise 1-(2)

Next, we calculated the regression line. The x values are arrays created from the radius data, while y is calculated from the above x and the calculated regression coefficients v=[a,b].

The regression line is created with weight against radius scatter plot. The code and the output is in Figure 2.

```
# 回帰直線の計算
# Calculation of the regression line

# 例：radiusの最小値から最大値までの範囲を0.1刻みでカウントしたリストないしはnp.arrayを作成(np.arangeを使うと良い)\n
# Example: Create a list or np.array counting from the minimum to maximum
# value of 'radius' in increments of 0.1 (using np.arange is recommended)

x = np.arange(data['radius'].min(),
              data['radius'].max() + 0.1,
              0.1)

#上のxと計算した回帰直線の係数v=[a,b]よりyを計算（numpyの列ベクトルのアクセスの仕方に注意）
# Calculate y from the above x and the calculated regression coefficients v=[a,b]
# (Pay attention to how to access numpy column vectors)

y = a * x + b

# 描画 (Plotting)
plt.close()
plt.scatter(data['radius'], data['weight'])
plt.plot(x, y)
plt.xlabel('radius')
plt.ylabel('weight')
plt.savefig('ex_1-2.png') # ex_1_sample.ipynbと同じディレクトリにグラフが保存される
plt.show()
```

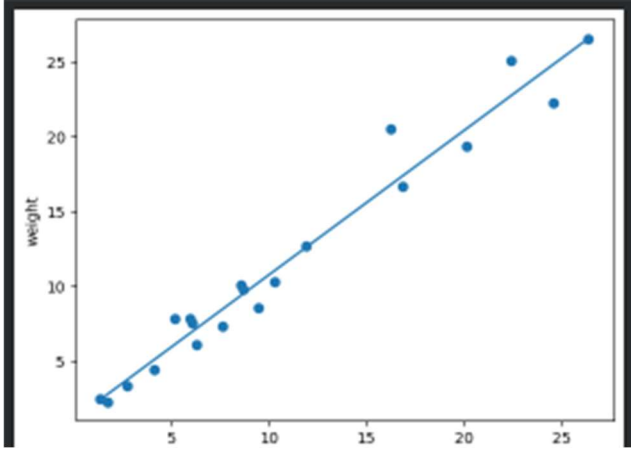


Figure 2 Exercise 1-(2)

Exercise 1-(3)

We created the code for the prediction process based on the fitted line, and output the predicted values and the graph.

The x-value coreesponding to the y-value we want to predict is $x_{predicted}$= 3.25

$y_{predicted}$= $ax_{predicted}$ +b

The code and the output graph is in Figure 3.



Figure 3 Exercise 1-(3)

Exercise 2

In Exercise 2, we learned how the least squares method becomes sensitive to outliers. We used dataset with outlier Mountain A. (Figure 4)

Exercise 2-(1)

Using the least squares method, fit a straight line of the form y=ax+b to this dataset, and report the values of a and b obtained.

We used the same method as in Exercise 1-(1). (Figure 5)

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
# 各山の生育状況のデータの読み込み
# 班に応じて読み込むデータセットを変える
# Loading growth data for each mountain
# Change the dataset to be loaded depending on your group

data_ir = pd.read_csv("/content/drive/MyDrive/Advanced_Lab/ex2/mountain_A_ir.csv", header=0, index_col=None)
data_ir
```

	radius	weight
0	1.35	2.45
1	1.72	20.33
2	2.70	3.34
3	4.13	4.44
4	6.29	6.11
5	5.92	7.83
6	7.62	7.33
7	9.48	8.52
8	8.65	9.80
9	10.30	10.31
10	5.16	7.81
11	6.05	7.52
12	8.57	10.06
13	11.90	12.64
14	16.84	16.64
15	16.24	20.50
16	20.15	19.36
17	24.62	22.25
18	22.45	25.07
19	26.40	5.50

Figure 4 Dataset for Exercise 2

```
# 2-1,2
# 課題1と同じ計算
# \Sigmaと\sigmaの計算を記述
# 2-1,2
# Same calculations as Exercise 1
# Describe the calculation of \Sigma and \sigma

# vの計算を記述
# Describe the calculation of v

x=data_ir['radius']
y=data_ir['weight']
N=len(x)

import numpy as np
Summation = np.array([
    [np.sum(x**2), np.sum(x)],
    [np.sum(x), N]])
sigma = np.array([
    np.sum(x * y),
    np.sum(y)
])
print("Summation(I):")
print(Summation)
print("\nSigma vector (σ):")
print(sigma)
a,b= np.linalg.inv(Summation) @ sigma
print("a=",a)
print("b=",b)
print("y=",a,"x+",b)
```

```
Summation(I):
[[3471.3212  216.54  ]
 [ 216.54     20.    ]]

Sigma vector (σ):
[3059.4869  227.81  ]
a= 0.5262385531708053
b= 5.692915184819691
y= 0.5262385531708053 x+ 5.692915184819691
```

Figure 5 Exercise 2-(1)

We obtained y= ax+b= 0.5262385531708053 x+ 5.69291518481969

Exercise 2-(2)

Plot the dataset and the fitted line y=ax+b.

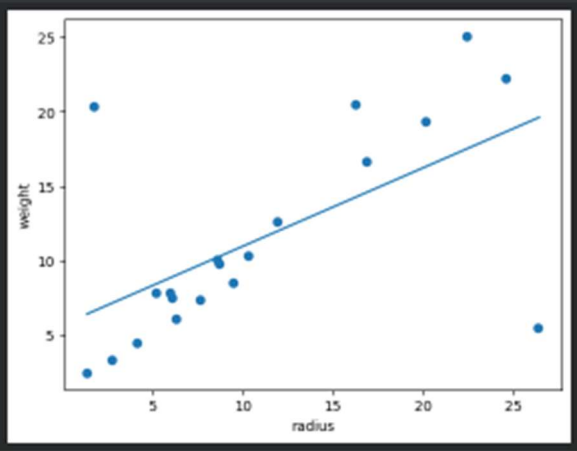
We obtained the following result.

```
# 回帰直線の計算 (Calculation of the regression line)
# 例：radiusの最小値から最大値までの範囲を0.1刻みでカウントしたリストないしはnp.arrayを作成(np.arangeを使うと良い)
# Example: Create a list or np.array counting from the minimum to maximum
# value of 'radius' in increments of 0.1 (using np.arange is recommended)\n

x = np.arange(data_ir['radius'].min(),
              data_ir['radius'].max() + 0.1,
              0.1)

# 上記のxと計算した回帰直線の係数v=[a,b]よりyを計算 (numpyの列ベクトルのアクセスの仕方に注意) \n
# Calculate y from the above x and the calculated regression coefficients v=[a,b]
# (Pay attention to how to access numpy column vectors)
y = a * x + b


# 描画
plt.close()
plt.scatter(data_ir['radius'], data_ir['weight'])
plt.plot(x, y)
plt.xlabel('radius')
plt.ylabel('weight')
plt.savefig('ex_2-2.png')
plt.show()
```



Exercise 2-(3)

Fill in the blanks for the prediction process based on the fitted line, and output the predicted values and the graph.Same procedure as Exercise 1-(3). Output is in Figure 6.

Exercise 2-(4)

Discuss the differences between the results obtained in Exercise 1 and Exercise 2. Furthermore, evaluate the degree of fitting for both Exercise 1 and Exercise 2. It is recommended to use the calculated sum of squared residuals as a reference for your discussion.

Difference between the results obtained in Exercise 1 and Exercise 2 is that the regression line is closer to the datapoints as exercise 1 had a smaller sum of squared residuals while exercise 2 had a greater sum of squared residuals so the regression line for exercise 2 was further away from the data points.

```
# 予測値の計算を記述 (Describe the calculation of the predicted value)
x_pred = 3.25
y_pred = a*x_pred+b
print('y_pred: %s' % y_pred)

# 描画 (Plotting)
plt.close()
plt.scatter(data_ir['radius'], data_ir['weight'], label='given')
plt.scatter(x_pred, y_pred, label='predicted')
plt.plot(x, y)
plt.legend()
plt.xlabel('radius')
plt.ylabel('weight')
plt.savefig('ex_2-3.png')
plt.show()

y_pred: 7.403190482624888
```

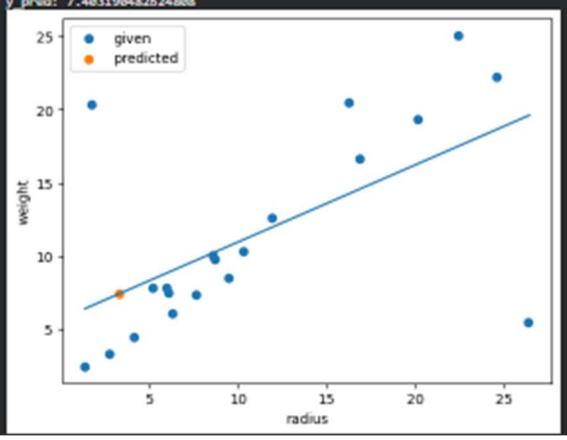


Figure 6 Code and output for Exercise 2-(3)

Exercise 3: Multi-Regression Analysis

Analysis of datasets with three or more variables are called multiple regression analysis.

First, we assume that the plane fitted to the dataset is $z = ax + by + c$. The objective function f minimized by the least squares method is

$$f(a,b,c) = \frac{1}{2}\sum_{i=1}^{N}(z_i - (ax_i + by_i + c))^2, \tag{1}$$

As in the case of two variables, the objective function f takes its minimum value when the partial derivatives with respect to each variable become zero.

$$\frac{\partial f}{\partial a} = \sum_{i=1}^{N}(z_i - ax_i - by_i - c)(-x_i) = a\sum_{i=1}^{N}x_i^2 + b\sum_{i=1}^{N}x_iy_i + c\sum_{i=1}^{N}x_i - \sum_{i=1}^{N}x_iz_i, \tag{2}$$

$$\frac{\partial f}{\partial b} = \sum_{i=1}^{N}(z_i - ax_i - by_i - c)(-y_i) = a\sum_{i=1}^{N}x_iy_i + b\sum_{i=1}^{N}y_i^2 + c\sum_{i=1}^{N}y_i - \sum_{i=1}^{N}y_iz_i, \tag{3}$$

$$\frac{\partial f}{\partial c} = \sum_{i=1}^{N}(z_i - ax_i - by_i - c)(-1) = a\sum_{i=1}^{N}x_i + b\sum_{i=1}^{N}y_i + c\sum_{i=1}^{N}1 - \sum_{i=1}^{N}z_i, \tag{4}$$

By setting each equation equal to 0 and arranging them in matrix form, we obtain

$$\begin{pmatrix} \sum_{i=1}^{N}x_i^2 & \sum_{i=1}^{N}x_iy_i & \sum_{i=1}^{N}x_i \\ \sum_{i=1}^{N}x_iy_i & \sum_{i=1}^{N}y_i^2 & \sum_{i=1}^{N}y_i \\ \sum_{i=1}^{N}x_i & \sum_{i=1}^{N}y_i & \sum_{i=1}^{N}1 \end{pmatrix}\begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^{N}x_iz_i \\ \sum_{i=1}^{N}y_iz_i \\ \sum_{i=1}^{N}z_i \end{pmatrix} \tag{5}$$

This yields the **normal equations**, and by solving them we obtain the optimal parameters a, b, c. Even when the number of variables is four or more, the normal equations can be derived in the same manner.

Exercise 3-(1)

To do Exercise 3, first we downloaded sample_ex3.csv. Then we uploaded the file using mount from drive to google colab and the program file is kadai3.ipynb.

Using multiple regression analysis, we fit a plane of the form z=ax+by+c and obtained the values of a, b, and c.

```
data = pd.read_csv("/content/drive/MyDrive/Advanced_Lab/ex3/sample_ex3.csv")
print(data)
```

```
          x        y         z
0   2.52541  3.33408   8.89526
1   0.47153  1.58862  -2.79739
2   2.26400  3.28524   8.46666
3   1.76777  2.04258   2.67027
4   0.20046  2.52423   0.88011
..      ...      ...       ...
95  2.01724  4.11355  10.22678
96  1.46136  0.76159  -4.53775
97  2.60758  4.86200  15.38339
98  2.96470  2.51995   6.03890
99  0.84939  4.58597  10.40406

[100 rows x 3 columns]
```

From the code in figure 7 to solve multi-regression analysis, we obtained a= 1.948054792211689, b=4.072713376930325 and c = -10.108108404510412

```
import numpy as np

# Assign columns
x = data.iloc[:,0]
y = data.iloc[:,1]
z = data.iloc[:,2]

# Σ
Sigma = np.array([
    [np.sum(x**2), np.sum(x*y), np.sum(x)],
    [np.sum(x*y), np.sum(y**2), np.sum(y)],
    [np.sum(x),    np.sum(y),    len(x)]
])

# σ
sigma = np.array([
    [np.sum(x*z)],
    [np.sum(y*z)],
    [np.sum(z)]
])

# v = Σ^(-1)σ
v = np.linalg.inv(Sigma) @ sigma

a = v[0,0]
b = v[1,0]
c = v[2,0]

print("a =", a)
print("b =", b)
print("c =", c)
```

```
a = 1.948054792211689
b = 4.072713376938325
c = -10.108108404510412
```

Figure 7 Exercise 3-(1)

Exercise 3-(2)

Consider the equation

Z= ax+by+c

The coefficients aa and bb appear to indicate how strongly weight (x) and height (y) influence the throwing distance. However, as we can see from the fitting results for the datasets above, simply changing the unit can drastically change the magnitude of the coefficients. To avoid the influence of such units, we perform data standardization seen in Figure 9.

After we perform data standardization, we obtain the values of a, b, and c

a= 1.3193660807073326

b= -0.35769055262145394

c= 9.96399357513218e-16

```python
import numpy as np

# Assign columns
x = data.iloc[:,0]
y = data.iloc[:,1]
z = data.iloc[:,2]

# Σ
Sigma = np.array([
    [np.sum(x**2), np.sum(x*y), np.sum(x)],
    [np.sum(x*y), np.sum(y**2), np.sum(y)],
    [np.sum(x),    np.sum(y),    len(x)]
])

# σ
sigma = np.array([
    [np.sum(x*z)],
    [np.sum(y*z)],
    [np.sum(z)]
])
# vの計算
#    v = \Sigma^{-1}\sigmaを記述(逆行列の計算はnumpyで用意されているのを使うと良い)
v = np.linalg.inv(Sigma)@sigma

a = v[0][0]
b = v[1][0]
c = v[2][0]


print("a=",a)
print("b=",b)
print("c=",c)
```

```
a= 0.9174108583845939
b= -0.25125346990585484
c= 25.800047341355253
```

```python
# 各山の生育状況のデータの読み込み
#   班に応じて読み込むデータセットを変える
data_1 = pd.read_csv("/content/drive/MyDrive/Advanced_Lab/ex3/ex3-2_dataset_2.csv")
print(data_1)
```

```
   weight  height  distance
0      39    1.52        24
1      40    1.53        23
2      50    1.68        29
3      38    1.56        22
4      57    1.69        36
```

```python
import numpy as np

# Assign columns
x = data_1["weight"]
y = data_1["height"]
z = data_1["distance"]
```

Figure 8 Exercise 3-2-(1)

```
import numpy as np

# Assign columns
x = data_1["weight"]
y = data_1["height"]
z = data_1["distance"]

# Σ
Sigma = np.array([
    [np.sum(x**2), np.sum(x*y), np.sum(x)],
    [np.sum(x*y), np.sum(y**2), np.sum(y)],
    [np.sum(x),    np.sum(y),    len(x)]
])

# σ
sigma = np.array([
    [np.sum(x*z)],
    [np.sum(y*z)],
    [np.sum(z)]
])
# vの計算
#    v = \Sigma^{-1}\sigmaを記述(逆行列の計算はnumpyで用意されているのを使うと良い)
v = np.linalg.inv(Sigma)@sigma

a = v[0][0]
b = v[1][0]
c = v[2][0]


print("a=",a)
print("b=",b)
print("c=",c)
```

```
a= 0.9174108583846792
b= -25.125346990570847
c= 25.800047341322056
```

```
x = weight_normal_1
y = height_normal_1
z = distance_normal_1

Sigma_normal_1 = np.array([
    [np.sum(x**2), np.sum(x*y), np.sum(x)],
    [np.sum(x*y), np.sum(y**2), np.sum(y)],
    [np.sum(x),    np.sum(y),    len(x)]
])

sigma_normal_1 = np.array([
    [np.sum(x*z)],
    [np.sum(y*z)],
    [np.sum(z)]
])


# v = Σ^{-1}σ
v = np.linalg.inv(Sigma_normal_1) @ sigma_normal_1

a = v[0][0]
b = v[1][0]
c = v[2][0]

print("a=",a)
print("b=",b)
print("c=",c)
```

```
a= 1.3193660807073326
b= -0.3576905526214539 4
c= 9.96399357513218e-16
```

Figure 9 Exercise 3-2-(2) standardized data

In Exercise 3-3, we also perform standardization on a data to see which factor has the strongest influence. (Figure 10)

Standardize the data.

2.Using the least squares method, obtain the coefficients: a', b', c', d

Since a′,b′,c′,d′ are coefficients obtained from standardized data, convert them into coefficients corresponding to the original data using the following formulas:

Standardize the data.

2.Using the least squares method, obtain the coefficients: a', b', c', d

Since a′,b′,c′,d′ are coefficients obtained from standardized data, convert them into coefficients corresponding to the original data using the following formulas:

$$a = (s(w)/s(x))a'$$
$$b = (s(w)/s(y))b'$$
$$c = (s(w)/s(z))c'$$
$$d = E(w) - s(w)(a'(E(x)/s(x)) + b'(E(y)/s(y)) + c'(E(z)/s(z)))$$

```
# Standardize the data

data['weight_normal'] = (data['weight'] - np.mean(data['weight'])) / np.std(data['weight'])

data['height_normal'] = (data['height'] - np.mean(data['height'])) / np.std(data['height'])

data['distance_normal'] = (data['distance'] - np.mean(data['distance'])) / np.std(data['distance'])

data['grip_normal'] = (data['grip'] - np.mean(data['grip'])) / np.std(data['grip'])

x = data['grip_normal']
y = data['height_normal']
z = data['weight_normal']
w = data['distance_normal']

Sigma = np.array([
    [np.sum(x**2), np.sum(x*y), np.sum(x*z), np.sum(x)],
    [np.sum(x*y), np.sum(y**2), np.sum(y*z), np.sum(y)],
    [np.sum(x*z), np.sum(y*z), np.sum(z**2), np.sum(z)],
    [np.sum(x),   np.sum(y),   np.sum(z),   len(x)]
])

sigma = np.array([
    [np.sum(x*w)],
    [np.sum(y*w)],
    [np.sum(z*w)],
    [np.sum(w)]
])

# vの計算
#    v = \Sigma^{-1}\sigmaを記述(逆行列の計算はnumpyで用意されているのを使うと良い)

v_normal = v = np.linalg.inv(Sigma)@sigma



print(v_normal)
```

```
[[ 3.47809462e-01]
 [ 3.06088708e-01]
 [ 2.81039177e-01]
 [-2.23837619e-17]]
```

Thus, we obtained $\begin{pmatrix} a\prime \\ b\prime \\ c\prime \\ d \end{pmatrix} = \begin{pmatrix} 3.47809462e-01 \\ 3.06088708e-01 \\ 2.81039177e-01 \\ -2.23837619e-17 \end{pmatrix}$

Week 2: Steepest Descent Method and Neural Network

The second week of the laboratory course centred on optimization techniques, nonlinear data fitting, and the fundamentals of machine learning. In the first step, we learned the theory behind the steepest descent optimization method and applied the provided sample programs to complete Exercise 4. The second step introduced nonlinear curve fitting while providing a basic understanding of neural networks. Expanding on these concepts, the third step involved studying neural network programming in greater depth and completing Exercise 5, which included independent research on the issue of overfitting. In the fourth step, we explored deep learning by learning how to perform data classification with TensorFlow and carrying out Exercise 6 using MNIST data of handwritten characters.

Steepest Descent Method

This method substitutes an appropriate initial value into the decision variable and repeatedly updates the value to obtain the optimal objective value.

For the problem of finding the value of a that minimizes the objective function,

the update rule of the steepest descent method is

$$a^{(k+1)} = a^{(k)} - \alpha f'(a^{(k)})$$

Here, a(k) is the value of the decision variable a obtained in the k-th iteration, and f′(a(k)) is the derivative of the objective function with respect to a evaluated at a = a(k). The parameter α is a small positive constant that controls how much the variable is updated in each iteration.

The basic idea of the steepest descent method is to gradually approach the optimal value by slightly changing the variable in the direction opposite to the derivative. The derivative represents the slope of the graph.

Exercise 4

We did gradient descent of the objective function

$$f(a) = (a-2)^2 * (a+2)^2$$

We then modified some parameters: varying the learning rate (step size) and explain the relationship between the convergence speed and the accuracy of the solution.

```python
# Define objective function
def f(a):
    return (a-2)**2*(a+2)**2


# Define derivative
def grad_f(a):
    return 4*a*(a**2-4)


# Parameters
a = -1.0     # initial value
alpha = 0.01     # learning rate
max_iter = 100


# Store values
a_values = []
f_values = []
grad_values = []


# Steepest descent iteration
for i in range(max_iter):

    a_values.append(a)
    f_values.append(f(a))
    grad_values.append(grad_f(a))

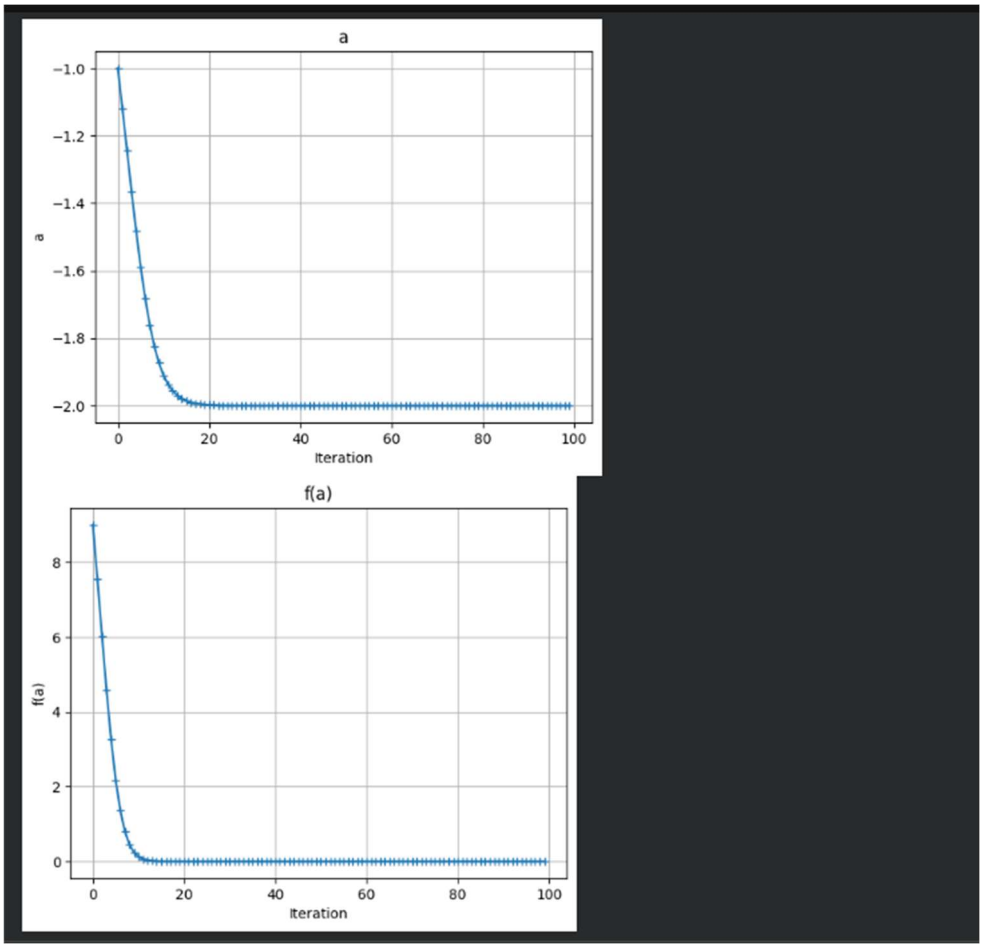
    # Update a
    a = a - alpha * grad_f(a)


print("Optimal a =", a)
print("Minimum f(a) =", f(a))

plt.plot(a_values, marker="+")
plt.grid()
plt.title("a")
plt.xlabel("Iteration")
plt.ylabel("a")
plt.show()

plt.plot(f_values, marker="+")
plt.grid()
plt.title("f(a)")
plt.xlabel("Iteration")
plt.ylabel("f(a)")
plt.show()
```

```
Optimal a = -1.9999999999999998
Minimum f(a) = 7.888609052210118e-31
```



We then varied different parameters of a(initial value), alpha (learning rate) and max number of iterations =100

When the alpha learning rate is huge such as 0.5, the objective function becomes oscillatory instead of decreasing smoothly.

```
a=3.0
alpha=0.5
Iter=100
```

As a result, convergence becomes unstable and the final accuracy decreases.

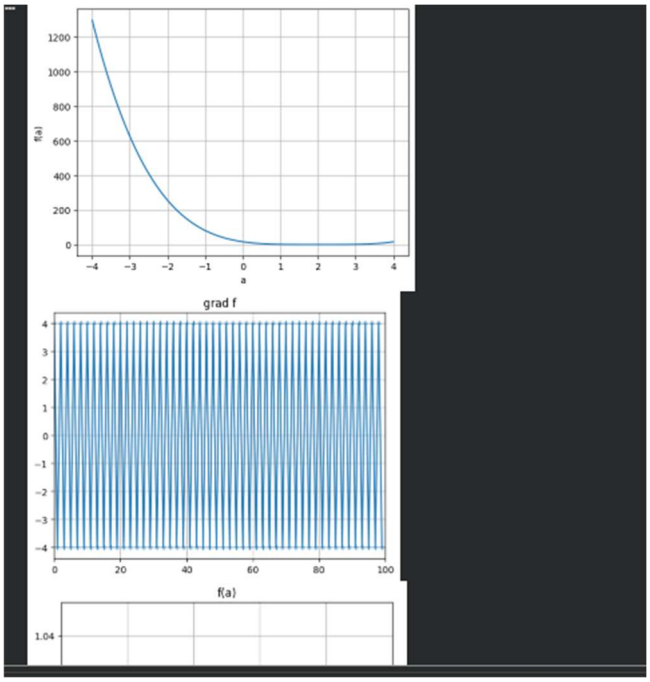


Figure 10 When learning rate alpha=0.5

The learning rate α controls the trade-off between convergence speed and solution accuracy:

- Small α → slow convergence but high accuracy.
- Moderate α → fastest stable convergence with high accuracy (optimal).
- Large α → faster initial progress but reduced accuracy due to overshooting.
- Very large α → unstable updates and divergence.

Therefore, an appropriately chosen α provides the best balance between convergence speed and accuracy, while values that are too small or too large degrade optimization performance.

Exercise 5: Neural Network

A neural network was trained on data points from the sin function.

For the training, we used the parameters

パラメータ（Parameters）

- num_hidden: 隠れ層のパーセプトロン数（Number of perceptrons in the hidden layer）
- max_loop: ループ数（Number of loops (iterations)）
- lr: 学習率（Learning rate）

```
t1 = time.time()
NUM_HIDDEN = 128
MAX_LOOP = 500
lr = 0.005
NUM_INPUT = 1
NUM_OUTPUT = 1

# 重み初期化
# Initialize weights

w1 = np.random.randn(NUM_HIDDEN, NUM_INPUT)
b1 = np.random.randn(NUM_HIDDEN, 1)
w2 = np.random.randn(NUM_OUTPUT, NUM_HIDDEN)
b2 = np.random.randn(NUM_OUTPUT, 1)

# loss 保存用
# Storage for loss history

train_losses = np.zeros(MAX_LOOP)
test_losses = np.zeros(MAX_LOOP)

def get_delta():
    pass

for loop in range(MAX_LOOP):
    for n in range(num_train):
        x = x_train[n]
        t = t_train[n]

        z = np.zeros(NUM_HIDDEN)
        y = np.zeros(NUM_OUTPUT)
        a = np.zeros(NUM_HIDDEN)

        d1 = np.zeros(NUM_HIDDEN)
        d2 = np.zeros(NUM_OUTPUT)

        #### 課題5-1（ここから）####
        #### Ex 5-1 (Start) ####

        # 1層目の計算(x -> z)
        # Calculation for the 1st layer (x -> z)

        for j in range(NUM_HIDDEN):
            for i in range(NUM_INPUT):
                a[j] += w1[j, i] * x
            a[j] += b1[j]
            z[j] = np.tanh(a[j])


        # 2層目の計算(z -> y)
        # Calculation for the 2nd layer (z -> y)

        for k in range(NUM_OUTPUT):
            for j in range(NUM_HIDDEN):
                y[k] += w2[k, j] * z[j]
            y[k] += b2[k]

        #### 課題5-1（ここまで）####
```

We also modified the code so that it shows the Mean Squared Error

```
    # NNforward計算関数を使うと便利
    #### Ex 5-2 ####
    # In the following train_loss and test_loss sections, currently set to 'None',
    # modify the code so that these are calculated as the Mean Squared Error (MSE).
    # Using the 'nn_forward' calculation function will be convenient.

    pred_train = np.array([nn_forward(x, w1, b1, w2, b2).item() for x in x_train])
    pred_test = np.array([nn_forward(x, w1, b1, w2, b2).item() for x in x_test])

    train_loss = 1/num_train * np.sum((t_train - pred_train)**2)
    test_loss = 1/num_test * np.sum((t_test - pred_test)**2)

    train_losses[loop] = train_loss
    test_losses[loop] = test_loss

    # 全てコーディング終了したらコメント化解除
    # Uncomment the line below once all coding is finished

    print("loop:{:>3}\t train loss:{:.4f}\t test loss:{:.4f}".format(loop, train_loss, test_loss))



    # 処理後の時刻
    # Display elapsed time
t2 = time.time()

# 経過時間を表示
elapsed_time = t2-t1
print(f"経過時間：{elapsed_time}")
```

The first 10 lines of the output shows train loss for each loop

```
loop:  0         train loss:0.3366       test loss:0.4221
loop:  1         train loss:0.2446       test loss:0.2904
loop:  2         train loss:0.1822       test loss:0.2052
loop:  3         train loss:0.1386       test loss:0.1493
loop:  4         train loss:0.1078       test loss:0.1120
loop:  5         train loss:0.0858       test loss:0.0868
loop:  6         train loss:0.0698       test loss:0.0696
loop:  7         train loss:0.0579       test loss:0.0576
loop:  8         train loss:0.0491       test loss:0.0493
loop:  9         train loss:0.0423       test loss:0.0433
loop: 10         train loss:0.0370       test loss:0.0389
```

The final result of Exercise 5(kadai 5) is in Figure 11.

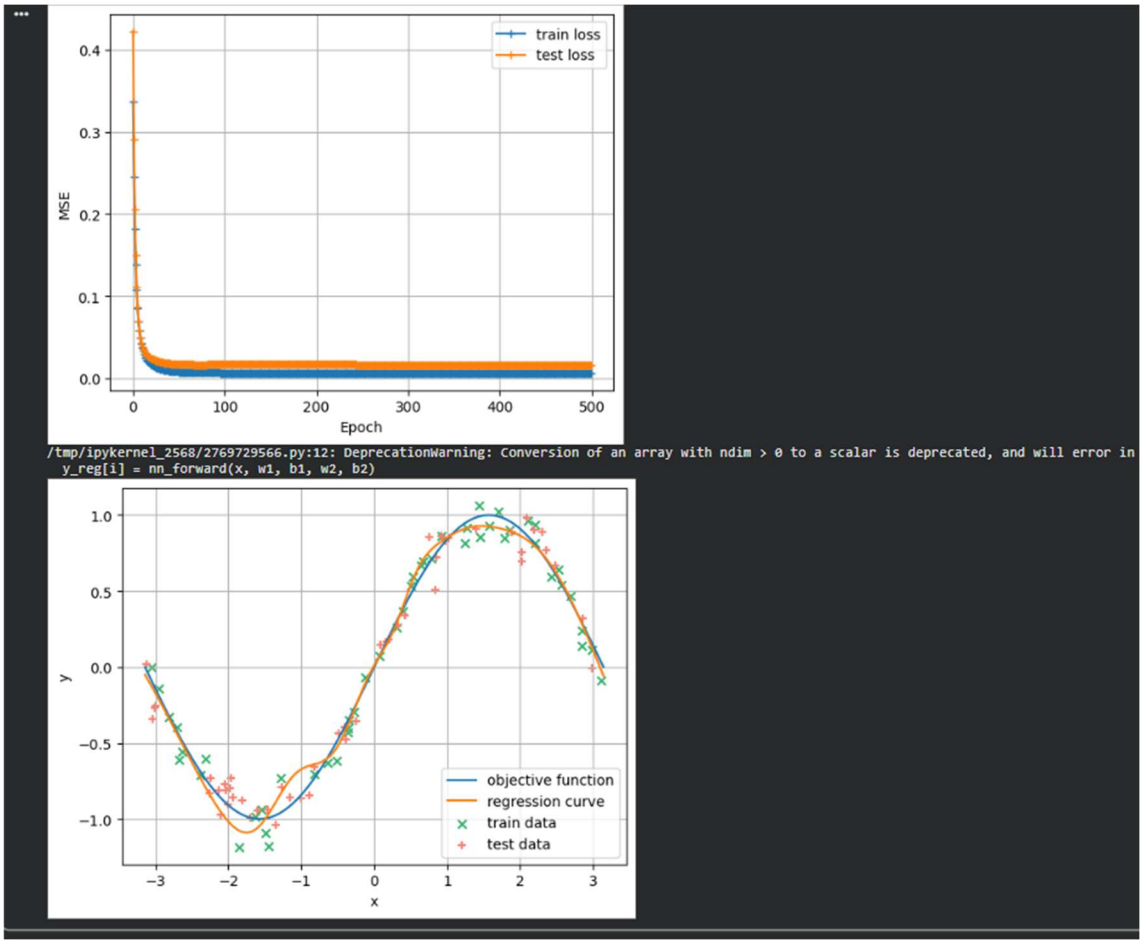


Figure 11 Result of Exercise 5

Exercise 6

We used tensorflow to train a keras model on existing handwriting data from MNIST. We modified the parameters to increase the accuracy and decrease loss.

- perception_size: パーセプションの数（Number of perceptrons）
- layer_size: 層の数（Number of layers）
- activation: 活性化関数（Activation function）
- optimizer: 最適化アルゴリズム(reference を参照のこと)（Optimization algorithm (refer to the reference section)）
- alpha: 正則化項係数（Regularization parameter coefficient）
- max_iter: 学習回数（Number of training iterations）

- batch_size: バッチのサイズ（Size of the mini-batch）
- shuffle: データをシャッフルするかどうか（Whether or not to shuffle the data）

We used the following parameters to increase the accuracy

```
perception_size = 128
layer_size = 3
activation = 'relu'
optimizer = tf.keras.optimizers.SGD()
max_iter = 30
alpha = 0.001
batch_size = 32
shuffle = 'true'

# learnig_rate, power_t, tol, early_stopping, validation_fractionは未実装
learning_rate = 0.001
power_t = 0.5
tol = 1e-4
early_stopping = True
validation_fraction = 0.1
```

learning_rate	Controls the size of weight updates	Faster learning but risk of instability	Slower, more stable learning	**0.001 (Adam)**
power_t	Controls how quickly the learning rate decays	Learning rate decreases faster; training becomes more conservative	Learning rate stays higher for longer	**0.5**
tol	Minimum improvement required to continue training	Stops training sooner	Trains longer to capture smaller improvements	**1e-4**
early_stopping	Stops training when validation performance plateaus	Prevents overfitting and saves time	If disabled, training always runs for all epochs	**True**
validation_fraction	Fraction of training data reserved for validation	More reliable validation but fewer training samples	More training data but less reliable validation	**0.1 (10%)**

The final result of Exercise 6 is in Figure 12. After 30 iterations, the accuracy is 0.9776 and loss is 0.1995.

Parameter	Recommended Value	Purpose	Effect if Increased
`perception_size`	128	Number of neurons per hidden layer	More capacity, higher accuracy (up to a point)
`layer_size`	2	Number of hidden layers	Learns more complex features
`activation`	`'relu'`	Activation function	ReLU converges quickly
`optimizer`	`Adam(learning_rate=0.001)`	Optimization algorithm	Faster convergence than SGD
`max_iter`	20	Number of epochs	More training until convergence
`alpha`	1e-4	L2 regularization strength	Smaller values reduce underfitting
`batch_size`	32	Mini-batch size	Good balance of speed and accuracy
`shuffle`	True	Shuffle training data	Prevents learning order bias
`learning_rate`	0.001	Step size for weight updates	Too high → unstable, too low → slow learning
`power_t`	0.5	Learning rate decay exponent (only used with `learning_rate='invscaling'` in SGD)	Larger values decrease the learning rate faster
`tol`	1e-4	Minimum improvement before stopping	Smaller values train longer
`early_stopping`	True	Stop when validation accuracy no longer improves	Reduces overfitting
`validation_fraction`	0.1	Fraction of training data used for	Typical choice is 10%

- Accuracy generally improves up to 2–3 layers for RELU.
- Beyond that, returns diminish for MNIST.

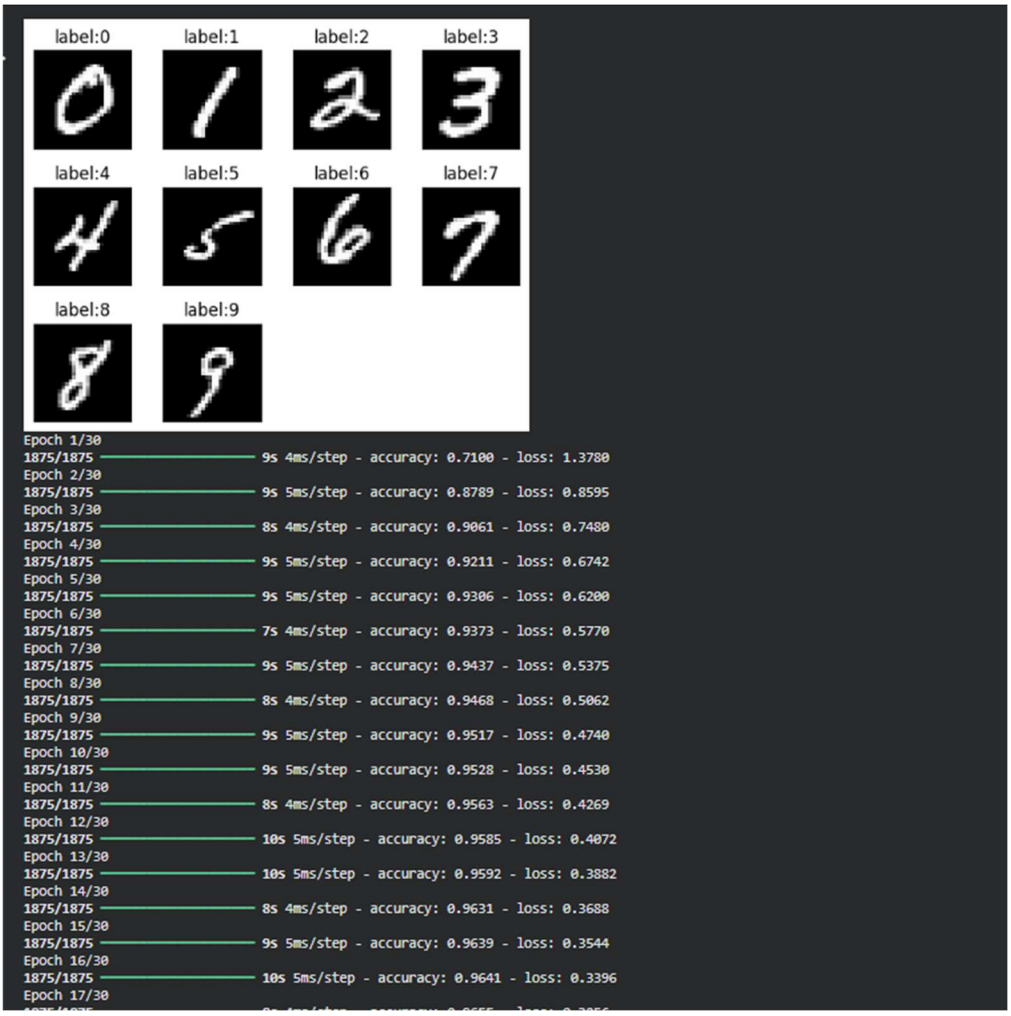


Figure 12 Final result of Exercise 6

Week 3: Combinatorial Problems and Genetic Algorithm

Exercise 8: Optimisation of the Knacksack Problem via Genetic Algorithm

Exercise 8-1: Sample Code

During the third week of the laboratory course, our activities centered on understanding and applying evolutionary computation methods. In Step 1, we learned the fundamental theory behind Genetic Algorithms (GAs), using the knapsack problem to illustrate their basic principles. Step 2 focused on developing and implementing a simple Genetic Algorithm in code. After establishing these fundamentals, Step 3 involved applying a Genetic Algorithm to solve the partition problem as part of Exercise 8. In Step 4, we extended our understanding by studying how Genetic Algorithms can be adapted to address the Traveling Salesman Problem (TSP). This was followed by Step 5, where we implemented the algorithm to complete Exercise 9. At the conclusion of the laboratory

session, each group selected and confirmed a topic for the Week 4 free project. These topics were then posted in the general Microsoft Teams channel to prevent duplication among groups before beginning the final project.

We choose the hyperparamter settings below, along with the output

Hyperparameter Settings

+ Code + Text

```
# GAのハイパーパラメータ（# GA Hyperparameters）
# 世代交替の回数（# Number of generations (iterations)）
n_generation = 100

# 遺伝子の数（# Population size (number of chromosomes)）
n_chromosome = 100

# 突然変異率（# Mutation rate）
p_mutation = 0.05

# 交叉させる遺伝子の数（初期実装は全遺伝子の50%を交叉）
# Number of chromosomes to undergo crossover (Default: 50%)
n_cross = int(n_chromosome * 0.5)

# エリート選抜で生き残る遺伝子の数（初期実装は全遺伝子の60%が生存し，ほかは上書きされる）
# Number of elite chromosomes to survive
n_selection = int(n_chromosome * 0.6)

# お菓子データ/ハイパーパラメータ（# Snack Data / Environment Parameters）
data_food = pd.read_csv('/content/drive/MyDrive/Advanced_Lab/ex8-1/data_set100.csv', index_col=None, header=0) # お菓子データの読み込み
n_food = len(data_food.index)                                                    # お菓子の数の取得
th_price = 1000                                                                  # 予算(超えてはいけない金額)
```

```
data_food
```

	cal	price
0	10.251506	1.075619
1	10.037979	0.842525
2	10.491804	9.663368
3	10.193108	9.625899
4	10.734684	6.431557
...	...	...
294	10.531546	8.352809
295	10.215215	2.257370
296	10.229019	1.509007
297	10.438709	6.117856
298	10.706157	7.524893

299 rows × 2 columns

The evaluation function calculates the total calories and price by reading the genes within a chromosome

```
# 評価関数（# Evaluation Function）
def eval_chromosome(chromosome):
    # chromosomeで選んでいるお菓子のみのデータをdata_foodから取得
    # Retrieve only the snack data selected by the chromosome
    tmp_food = data_food.loc[chromosome['gene']]

    # calとpriceで総和を取る
    # Calculate totals for calories (cal) and price
    chromosome['cal'] = tmp_food['cal'].sum()
    chromosome['price'] = tmp_food['price'].sum()

    # priceが予算を超えていたら総カロリーを0にする
    # If the price exceeds the budget, set total calories to 0 (penalty)
    if chromosome['price'] > th_price:
        chromosome['cal'] = 0

    return chromosome
```

The sample crossover function was uniform crossover.

Crossover Functions

```
# 1点交叉
# Single-point Crossover
#def cross_gene(gene1, gene2):

    # 交叉開始ポイントを乱数で生成
    # Generate a crossover point randomly
    i_point = np.random.randint(n_food)

    # 入れ替え
    # Swap segments
    tmp_gene1 = np.append(gene1[:i_point], gene2[i_point:])
    tmp_gene2 = np.append(gene2[:i_point], gene1[i_point:])

    return [tmp_gene1, tmp_gene2]
```

```
#def cross_gene_n(gene1, gene2, n=2):
    # 交叉開始ポイントを乱数で生成
    # Generate N crossover points randomly
    li_i_point = np.random.choice(n_food, size=n)
    li_i_point = sorted(li_i_point)

    # nポイント目までしか遺伝子が追加されないので，n_foodを追加
    # Append n_food to ensure the loop covers the end of the gene array
    li_i_point.append(n_food)

    # 交叉処理
    # Crossover processing
    tmp_gene1 = np.array([], dtype=bool)
    tmp_gene2 = np.array([], dtype=bool)
    is_crossover =False
    i_point_prev = 0
    for i_point in li_i_point:
        # is_crossoverでgene1とgene2の入れ替えを制御
        # Control the swapping of gene1 and gene2 using is_crossover toggle
        if is_crossover:
            tmp_gene1 = np.append(tmp_gene1, gene1[i_point_prev: i_point])
            tmp_gene2 = np.append(tmp_gene2, gene2[i_point_prev: i_point])
        else:
            tmp_gene1 = np.append(tmp_gene1, gene2[i_point_prev: i_point])
            tmp_gene2 = np.append(tmp_gene2, gene1[i_point_prev: i_point])

        is_crossover = not(is_crossover)
        i_point_prev = i_point

    return [tmp_gene1, tmp_gene2]
```

Mutation Function

```
def mutate_chromosome(chromosome):
    if np.random.rand() < p_mutation:
        i_gene = np.random.randint(n_food)
        chromosome['gene'][i_gene] = not(chromosome['gene'][i_gene])

    return chromosome
```

And the main genetic algorithm

Main Genetic Algorithm Loop

```
# 初期染色体の生成
# Initialize population (first generation)
chromosomes = [{
    'gene': np.random.choice([True, False], size=n_food),
    'cal': None,
    'price': None} for i in range(n_chromosome)]

# 世代ループ
# Generational Loop
acc_log = []
for i_generation in range(n_generation):
    # 染色体の評価
    # Evaluate chromosomes
    chromosomes = [eval_chromosome(chromosome) for chromosome in chromosomes]

    # calに基づいて染色体をソート
    #    reverse=True: 大きい順, reverse=False: 小さい順　にソート
    # Sort chromosomes based on fitness (calories)
    # reverse=True: Descending order (best fitness first)
    chromosomes = sorted(chromosomes, key=itemgetter('cal'), reverse=True)

    # 最優秀染色体の保存
    # Save the best chromosome of the current generation
    best_chromosome = copy.deepcopy(chromosomes[0])

    # エリート選抜
    # Elite Selection
    chromosomes = chromosomes[:n_selection] + [copy.deepcopy(best_chromosome) for i in range(n_chromosome - n_selection)]

    # 交叉
    # Crossover
    for i_cross in range(n_cross):
        # 交叉対象(親)の選択
        # Select parents for crossover
        #i_ch1, i_ch2 = select_chromosomes_roulette(chromosomes)
        i_ch1, i_ch2 = select_chromosomes(chromosomes)
        #tmp_genes = cross_gene(chromosomes[i_ch1]['gene'], chromosomes[i_ch2]['gene'])

        # 交叉
        # Perform crossover
        #tmp_genes = cross_gene_n(chromosomes[i_ch1]['gene'], chromosomes[i_ch2]['gene'])
        tmp_genes = cross_gene_uni(chromosomes[i_ch1]['gene'],chromosomes[i_ch2]['gene'])

        # tmpの内容を適用
        # Apply the new genes
        chromosomes[i_ch1]['gene'] = tmp_genes[0]
        chromosomes[i_ch2]['gene'] = tmp_genes[1]

    # 突然変異
    # Mutation
    chromosomes = [mutate_chromosome(chromosome) for chromosome in chromosomes]

    # 処理状態の表示
    # Progress Printout
    print('loop = %s, best_cal = %s' % (i_generation, best_chromosome['cal']))

    # カロリーの保存
    # Log best calorie value for plotting
    acc_log.append(best_chromosome['cal'])
```

Exercise 8-(2) is a class contest using the following CSV file number_set_100.csv containing 100 real numbers as the input for the partition problem. Divide the numbers into two sets so that the difference between the sums of the two sets is as small as possible. For the partition we obtain, we computed the score defined as the *reciprocal of the difference* between the total sums of the two sets.

Google Colab did not work in uploading the Excel file in my PC, so I used Jupyter Notebook in Anaconda to write the code.

The hyperparameter settings was set as the following

```
#ハイパーパラメータの設定
#Hyperparameter Settings
# GAのハイパーパラメータ（# GA Hyperparameters）
# ----------------------------
# GA Parameters
# ----------------------------
n_generation = 10000        # Number of generations
n_chromosome = 1000         # Population size
p_mutation = 0.005          # Mutation probability

elite_ratio = 0.20
n_elite = int(elite_ratio * n_chromosome)

restart = 20                # Number of GA runs
# お菓子データ/ハイパーパラメータ（# Snack Data / Environment Param'/content/drive/MyDrive/data_set100.csv', index_col=None, header=0) # お菓子データの
```

Evaluation Function

Replace the knapsack evaluation completely with

```
def create_chromosome():

    chromosome = {}

    chromosome["gene"] = np.random.choice(
        [True, False],
        size=n_number
    )

    chromosome["sum1"] = 0
    chromosome["sum2"] = 0
    chromosome["diff"] = 0
    chromosome["fitness"] = 0

    return chromosome
```

```
def eval_chromosome(chromosome):

    gene = chromosome["gene"]

    sum1 = np.sum(numbers[gene])
    sum2 = np.sum(numbers[~gene])

    diff = abs(sum1 - sum2)

    chromosome["sum1"] = sum1
    chromosome["sum2"] = sum2
    chromosome["diff"] = diff

    # Higher fitness is better
    chromosome["fitness"] = 1.0 / (diff + 1e-12)

    return chromosome
```

I then used Tournament selection which picks a random subset of candidates and pits them against each other, always choosing the best individual within that specific group. t usually converges much faster for partition problems.

```python
# --------------------------------
# Tournament Selection
# --------------------------------
def tournament_selection(chromosomes, k=5):

    candidates = np.random.choice(chromosomes, k, replace=False)

    best = candidates[0]

    for c in candidates:
        if c["diff"] < best["diff"]:
            best = c

    return copy.deepcopy(best)
```

Then uniform crossover was conducted, followed by mutation.

```python
# --------------------------------
# Uniform Crossover
# --------------------------------
def crossover(parent1, parent2):

    mask = np.random.rand(n_number) < 0.5

    child_gene = np.where(
        mask,
        parent1["gene"],
        parent2["gene"]
    )

    child = {
        "gene": child_gene,
        "sum1":0,
        "sum2":0,
        "diff":0,
        "fitness":0
    }

    return child
```

```python
# --------------------------------
# Mutation
# --------------------------------
def mutate(chromosome):

    gene = chromosome["gene"].copy()

    for i in range(n_number):

        if np.random.rand() < p_mutation:
            gene[i] = ~gene[i]

    chromosome["gene"] = gene

    return chromosome
```

Local Search: After mutation, the algorithm tries flipping every single number once. If moving one number to the opposite set improves the solution, it keeps that change. After applying local searh we also replaced the initial population.

```
# Local Search
# ------------------------------
def local_search(chromosome):

    chromosome = eval_chromosome(chromosome)

    improved = True

    while improved:

        improved = False

        current_best = chromosome["diff"]

        best_gene = chromosome["gene"].copy()

        for i in range(n_number):

            new_gene = chromosome["gene"].copy()

            new_gene[i] = ~new_gene[i]

            sum1 = np.sum(numbers[new_gene])
            sum2 = np.sum(numbers[~new_gene])

            diff = abs(sum1 - sum2)

            if diff < current_best:

                current_best = diff
                best_gene = new_gene
                improved = True

        chromosome["gene"] = best_gene
        chromosome = eval_chromosome(chromosome)

    return chromosome
```

```
]: chromosomes = []

for i in range(n_chromosome):

    c = create_chromosome()
    c = eval_chromosome(c)

    chromosomes.append(c)
```

Followed by the main Gentic Algorithm loop, broken into parts.



```
# ========================================
# Genetic Algorithm
# ========================================

best_solution = None
best_difference = float("inf")

history = []

for run in range(restart):

    print(f"Run {run+1}/{restart}")

    # ------------------------
    # Initial population
    # ------------------------
    chromosomes = []

    for i in range(n_chromosome):

        c = create_chromosome()
        c = eval_chromosome(c)

        chromosomes.append(c)

    run_history = []

    # ------------------------
    # Evolution
    # ------------------------
    for generation in range(n_generation):

        chromosomes.sort(key=lambda x: x["diff"])

        run_history.append(chromosomes[0]["diff"])

        # Perfect solution
        if chromosomes[0]["diff"] < 1e-4:
            break

        # ------------------------
        # Elitism
        # ------------------------
        new_population = []

        for i in range(n_elite):
            new_population.append(copy.deepcopy(chromosomes[i]))

        # ------------------------
        # Generate children
        # ------------------------
        while len(new_population) < n_chromosome:

            parent1 = tournament_selection(chromosomes)
            parent2 = tournament_selection(chromosomes)

            child = crossover(parent1, parent2)

            child = mutate(child)

            child = eval_chromosome(child)

            child = local_search(child)
```

Figure 13 Genetic Algorithm Main Loop Part 1

```
        # ------------------------
        # Generate children
        # ------------------------
        while len(new_population) < n_chromosome:

            parent1 = tournament_selection(chromosomes)
            parent2 = tournament_selection(chromosomes)

            child = crossover(parent1, parent2)

            child = mutate(child)

            child = eval_chromosome(child)

            child = local_search(child)

            new_population.append(child)

        chromosomes = new_population

    chromosomes.sort(key=lambda x: x["diff"])

    print("Best Difference =", chromosomes[0]["diff"])

    if chromosomes[0]["diff"] < best_difference:

        best_difference = chromosomes[0]["diff"]

        best_solution = copy.deepcopy(chromosomes[0])

        history = run_history
```

Figure 14 Genetic Algorithm Part 2

Compared with the original knapsack GA:

Original	**New**
Roulette selection	✅ Tournament selection
One-point crossover	✅ Uniform crossover
Single-bit mutation	✅ Multi-bit mutation
No improvement	✅ Local search (hill climbing)

The local search is especially effective for the partition problem because it can quickly reduce the difference by moving individual numbers between the two sets.

Finally, to generate the best partition

```
# ========================================
# Best Solution
# ========================================

best_solution = eval_chromosome(best_solution)

A1 = numbers[best_solution["gene"]]
A2 = numbers[~best_solution["gene"]]

print("\n========================")
print("BEST PARTITION FOUND")
print("========================")

print("Sum A1 =", best_solution["sum1"])
print("Sum A2 =", best_solution["sum2"])

print("Difference =", best_solution["diff"])

if best_solution["diff"] == 0:
    print("Score = Infinity")
else:
    print("Score =", 1 / best_solution["diff"])
```

The result is below, and using the formula Score=1/Difference, we obtained a score of 1,891,789

```
============================
BEST PARTITION FOUND
============================
Sum A1 = 25.224456083
Sum A2 = 25.224455554400002
Difference = 5.285999975512823e-07
Score = 1891789.6417564487
```

Exercise 9: The Travelling Salesman Problem Contest

For this, I also used Jupyter Notebook of Anaconda due to issues mounting the Excel File from Google Drive to Google Colab. I used the pandas read csv instead.

We set the hyperparameter settings as the following

```
32]: # ハイパーパラメータの設定 (Hyperparameters and settings)
     # GAのハイパーパラメータ (GA Hyperparameters)

     # 世代交替の回数
     # Number of generations (iterations)
     n_generation = 100

     # 遺伝子の数
     # Better parameters
     n_generation = 1000
     n_chromosome = 300

     p_mutation = 0.10

     n_cross = int(n_chromosome*0.8)
     n_elite = 20


     # 都市データ (# City Data)
     data_city = pd.read_csv(r'C:\Users\andre\OneDrive\Documents\IDE\3rdyearSpring\advanced labs II\tsp48.csv', header=None) # 都市データの読み込み
     n_city = len(data_city.index)                                    # 都市の数の取得 (# Get number of cities)
```

The evaluation function evaluates the chromosomes, calculate the distance between two cities, and store the calculated total Euclidean distance in the chromosome

```python
# 評価関数
# Evaluation Function

def eval_chromosome(chromosome):

    # 先頭から2都市取ってきて距離を計算
    # Take two cities from the start and calculate the distance

    dist_sum = 0
    for i_gene in range(n_city-1):
        city_from = chromosome['gene'][i_gene]
        city_to = chromosome['gene'][i_gene+1]

        # ユークリッド距離の計算
        # Calculate Eucl
        dist_sum += np.l                                    cy_from] - data_city.loc[city_to])

    # 染色体に計算したユークリッド距離の総和を格納
    # Store the calculated total Euclidean distance in the chromosome
    chromosome['dist'] = dist_sum

    return chromosome
```

Next, roulette selection followed by 1-point (city) crossover were employed.

```python
# ルーレット選択
# Roulette Selection
def select_chromosomes_roulette(chromosomes):

    # 全染色体の距離の総和
    # Total sum of the reciprocal of distances for all chromosomes
    sum_dist = sum([1 / x['dist'] for x in chromosomes])

    # 重み付きルーレットで選択
    # Selection using weighted roulette
    i_ch1 = np.random.choice(n_chromosome, p=[(1 / x['dist']) / sum_dist for x in chromosomes])
    i_ch2 = np.random.choice(n_chromosome, p=[(1 / x['dist']) / sum_dist for x in chromosomes])

    return [i_ch1, i_ch2]
```

交叉関数

```python
# 1点(都市)交叉
# 1-point (City) Crossover
def cross_gene(gene1, gene2):

    # 交叉ポイント(i_gene1 < i_gene2)
    # Crossover points (ensuring i_gene1 < i_gene2)
    points = np.random.choice(n_city, size=2, replace=False)
    points = sorted(points)

    for i in range(points[0], points[1]):

        # iに対応する都市
        # Cities corresponding to index i
        city1 = gene1[i]
        city2 = gene2[i]

        # city1の入れ替え
        # Swap city1
        gene1[gene1==city2], gene1[i] = city1, city2

        # city2の入れ替え
        # Swap city2
        gene2[gene2==city1], gene2[i] = city2, city1

    return [gene1, gene2]
```

We also replaced the mutation function with inversion mutation. For a 48-city TSP, inversion mutation is one of the most used mutation operators because it provides a good balance between exploration and preserving good routes.

```python
#ex 9 start

def mutate_chromosome(chromosome):

    if np.random.rand() < p_mutation:

        i, j = sorted(np.random.choice(n_city,2,replace=False))

        chromosome["gene"][i:j] = chromosome["gene"][i:j][::-1]
    return chromosome


    # 課題9穴埋めここまで
    # Ex 9: Fill in ending here

    return chromosome
```

The Genetic Main Loop Algorithm was then run which for each iteration(epoch) calculated the best distance.

Genetic Algorithm Main Loop

```python
[ ]: # 初期染色体の生成
# Generate initial chromosomes

chromosomes = [{
    'gene': np.random.choice(data_city.index, size=n_city, replace=False),
    'dist': None} for i in range(n_chromosome)]


# 世代ループ
# Generation Loop

acc_log = []
for i_generation in range(n_generation):

    # 染色体の評価
    # Evaluate chromosomes
    chromosomes = [eval_chromosome(chromosome) for chromosome in chromosomes]

    # distに基づく染色体のソート
    # Sort chromosomes based on distance (dist)
    chromosomes = sorted(chromosomes, key=itemgetter('dist'), reverse=False)

    # 最優秀染色体の保存
    # Save the best chromosome
    best_chromosome = copy.deepcopy(chromosomes[0])

    # エリート選抜
    # Elite Selection
    chromosomes = chromosomes[:n_selection] + [copy.deepcopy(best_chromosome) for i in range(n_chromosome - n_selection)]

    # 交叉
    # Crossover

    for i_cross in range(n_cross):

        # 交叉対象(親)の選択
        # Select targets (parents) for crossover
        i_ch1, i_ch2 = select_chromosomes_roulette(chromosomes)

        # 交叉
        # Perform Crossover
        tmp_genes = cross_gene(chromosomes[i_ch1]['gene'], chromosomes[i_ch2]['gene'])

        # tmpの内容を適用
        # Apply the results from tmp
        chromosomes[i_ch1]['gene'] = tmp_genes[0]
        chromosomes[i_ch2]['gene'] = tmp_genes[1]

    # 突然変異
    # Mutation
    chromosomes = [mutate_chromosome(chromosome) for chromosome in chromosomes]

    # 進捗表示
    # Progress Display
    print('loop = %s, best_dist = %s' % (i_generation, best_chromosome['dist']))

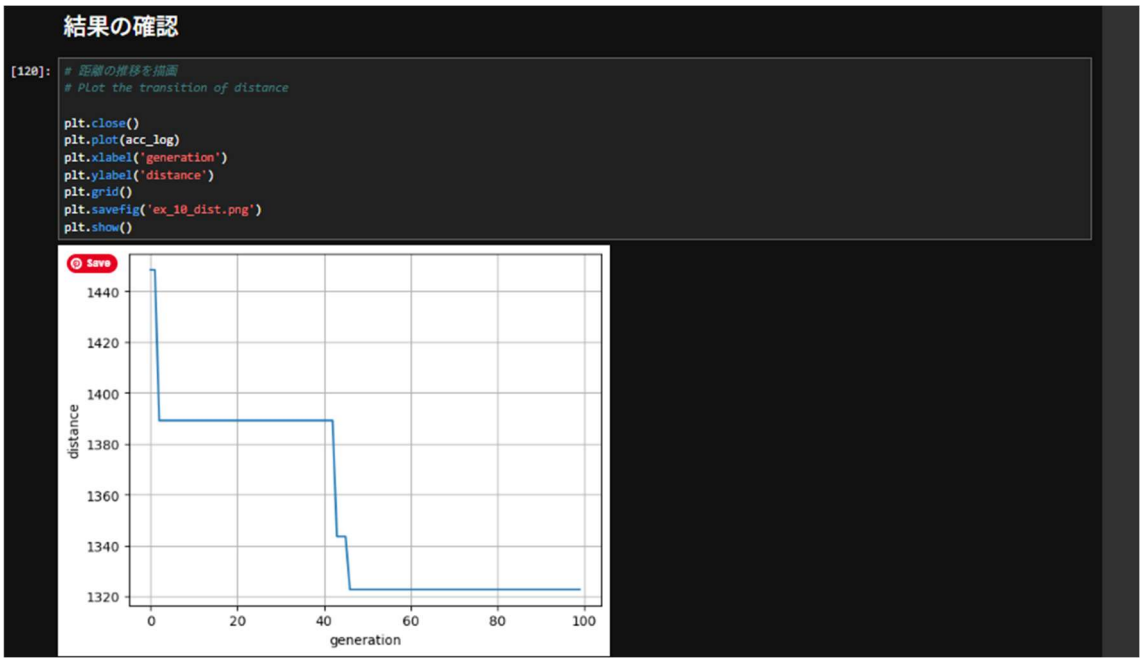
    # distanceの保存
    # Save distance history
    acc_log.append(best_chromosome['dist'])
```

The best distance was 34876.05588539103.

```
loop = 682, best_dist = 34876.05588539103
loop = 683, best_dist = 34876.05588539103
loop = 684, best_dist = 34876.05588539103
loop = 685, best_dist = 34876.05588539103
loop = 686, best_dist = 34876.05588539103
loop = 687, best_dist = 34876.05588539103
loop = 688, best_dist = 34876.05588539103
loop = 689, best_dist = 34876.05588539103
loop = 690, best_dist = 34876.05588539103
loop = 691, best_dist = 34876.05588539103
loop = 692, best_dist = 34876.05588539103
loop = 693, best_dist = 34876.05588539103
loop = 694, best_dist = 34876.05588539103
loop = 695, best_dist = 34876.05588539103
loop = 696, best_dist = 34876.05588539103
loop = 697, best_dist = 34876.05588539103
loop = 698, best_dist = 34876.05588539103
loop = 699, best_dist = 34876.05588539103
loop = 700, best_dist = 34876.05588539103
loop = 701, best_dist = 34876.05588539103
loop = 702, best_dist = 34876.05588539103
loop = 703, best_dist = 34876.05588539103
loop = 704, best_dist = 34876.05588539103
loop = 705, best_dist = 34876.05588539103
```

The code for printing the best distance did not work correctly, so I inferred from the last values of the loops.

The transition of the distance is below



While the route plot is in Figure

```python
# 経路の描画
# geneだけでは最初の都市に戻らないので，最後尾に追加
route = np.append(best_chromosome['gene'], best_chromosome['gene'][0])

plt.close()

# ルート(線)の描画
# Draw the route (lines)
plt.plot(data_city[0].loc[route], data_city[1].loc[route])


# 都市(点)の描画
# Draw the cities (points)
plt.scatter(data_city[0], data_city[1])


# グリッドの表示
# Display grid
plt.grid()

# 画像の保存と表示
# Save and show the image
plt.savefig('ex_10_map.png')
plt.show()
```

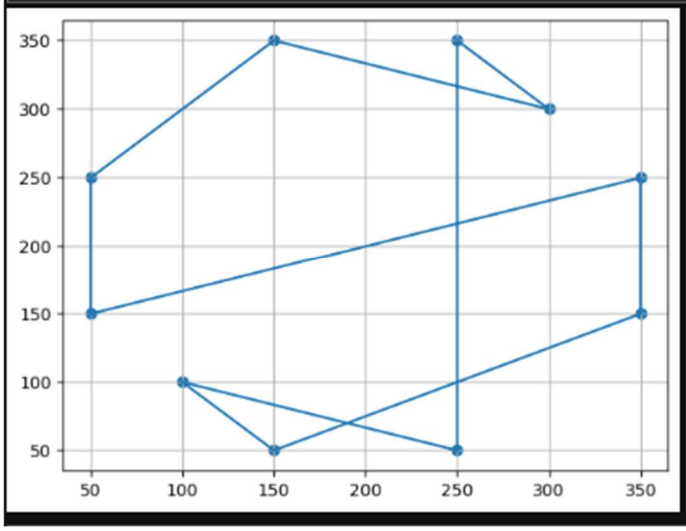


Figure 15 Route Plot of Task/Exercise 9

Week 4 Free Themed Project is next page in page 37.

Week 4 Free Theme Project for Real Life Optimization Project

Our group Group D choose the free themed project Apartment Recommendation System for Students Around University of Tsukuba.

Project Title
Finding the Best Apartment Around the University of Tsukuba Using Multi-Objective Optimization

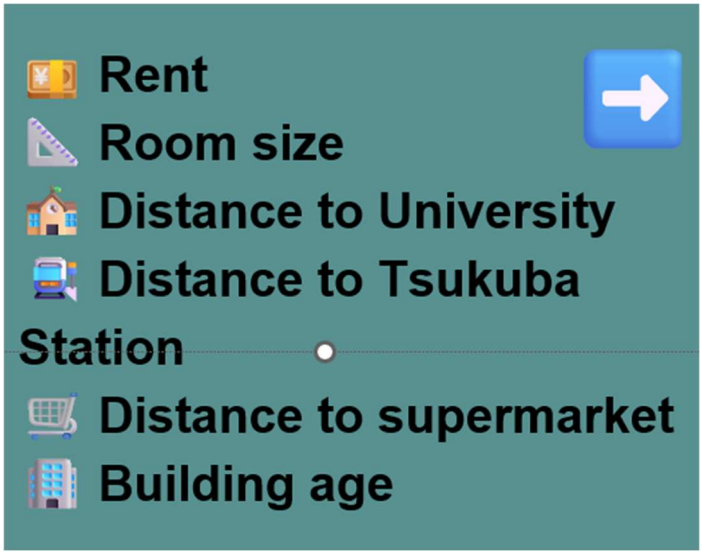
Problem to Solve
Students looking for apartments near the University of Tsukuba must consider multiple factors such as rent, room size, and convenience. Comparing many apartments manually is difficult.

Goal
Develop a system that recommends the most suitable apartment based on multiple criteria while balancing cost and convenience.

Method

- Collect apartment data around the University of Tsukuba (e.g., 50–150 listings).
- Evaluate apartments using factors such as rent, room size, distance to campus, distance to Tsukuba Station, and distance to supermarkets. Evaluate how important or weight is on each factor to determine the best apartment around the University of Tsukuba.
- Apply a Genetic Algorithm (or another multi-objective optimization method) to identify the best apartment recommendations.

We used the following factors:



First, we collected apartment data from Suumo for apartments in University of Tsukuba using web.

Then, we created a User Interface for people to input in the factors User inputs (Budget, Room size, Importance of Each factor from 1-5). Then we calculated the weight of the user rated scores(convert importance ratings into weights) and will use it for the Genetic Algorithm.

Weight= Score/ Total Score

```
import pandas as pd

from IPython.display import display
# ========================================
# Read Apartment Dataset
# ========================================

# CSV file
df = pd.read_excel("/content/drive/MyDrive/Advanced_Lab/take2.xlsx")

print("Apartment Dataset Loaded Successfully!")
display(df.head())

# ========================================
# User Interface
# ========================================

print("\n========== Apartment Preference ==========\n")

max_budget = float(input("Maximum budget (¥): "))
min_room_size = float(input("Minimum room size (m²): "))

importance_rent = int(input("Importance of Rent (1-5): "))
importance_room = int(input("Importance of Room Size (1-5): "))
importance_university = int(input("Importance of University Distance (1-5): "))
importance_supermarket = int(input("Importance of Supermarket Distance (1-5): "))
importance_station = int(input("Importance of Station Distance (1-5): "))
importance_age = int(input("Importance of Building Age (1-5): "))

# ========================================
 # Convert Importance Ratings into Weights
# ========================================

total = (
    importance_rent +
    importance_room +
    importance_university +
    importance_supermarket +
    importance_station +
    importance_age
)

weights = {
    "Rent": importance_rent / total,
    "Room Size": importance_room / total,
    "University Distance": importance_university / total,
    "Supermarket Distance": importance_supermarket / total,
    "Station Distance": importance_station / total,
    "Building Age": importance_age / total
}

print("\n========== Normalized Weights ==========")

for key, value in weights.items():
    print(f"{key:25s}: {value:.3f}")
```

For a sample, we input the

Maximum budget (¥): 50000

Minimum room size (m²): 30

Importance of Rent (1-5): 3

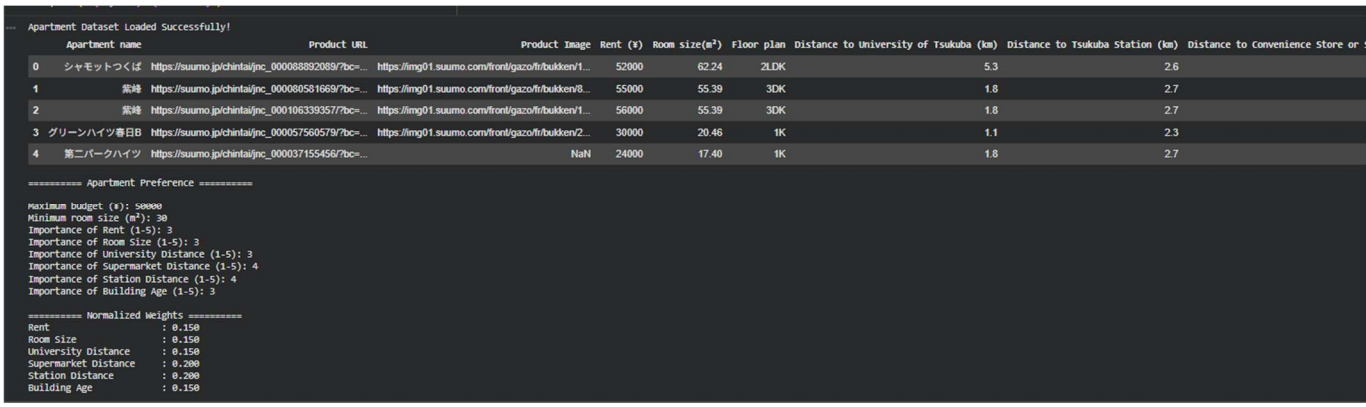
Importance of Room Size (1-5): 3

Importance of University Distance (1-5): 3

Importance of Supermarket Distance (1-5): 4

Importance of Station Distance (1-5): 4

Importance of Building Age (1-5): 3



```
Apartment Dataset Loaded Successfully!
```

	Apartment name	Product URL	Product Image	Rent (¥)	Room size(m²)	Floor plan	Distance to University of Tsukuba (km)	Distance to Tsukuba Station (km)	Distance to Convenience Store or
0	シャモットつくば	https://suumo.jp/chintai/jnc_000088892089/?bc=...	https://img01.suumo.com/front/gazo/fr/bukken/1...	52000	62.24	2LDK	5.3	2.6	
1	紫峰	https://suumo.jp/chintai/jnc_000080581669/?bc=...	https://img01.suumo.com/front/gazo/fr/bukken/8...	55000	55.39	3DK	1.8	2.7	
2	紫峰	https://suumo.jp/chintai/jnc_000106339357/?bc=...	https://img01.suumo.com/front/gazo/fr/bukken/1...	56000	55.39	3DK	1.8	2.7	
3	グリーンハイツ春日B	https://suumo.jp/chintai/jnc_000057560579/?bc=...	https://img01.suumo.com/front/gazo/fr/bukken/2...	30000	20.46	1K	1.1	2.3	
4	第二パークハイツ	https://suumo.jp/chintai/jnc_000037155456/?bc=...	NaN	24000	17.40	1K	1.8	2.7	

```
========== Apartment Preference ==========

Maximum budget (¥): 50000
Minimum room size (m²): 30
Importance of Rent (1-5): 3
Importance of Room Size (1-5): 3
Importance of University Distance (1-5): 3
Importance of Supermarket Distance (1-5): 4
Importance of Station Distance (1-5): 4
Importance of Building Age (1-5): 3

========== Normalized Weights ==========
Rent                : 0.150
Room Size           : 0.150
University Distance : 0.150
Supermarket Distance: 0.200
Station Distance    : 0.200
Building Age        : 0.150
```

After normalising the data,we use fitness function

$$J = \Sigma(RentScore \cdot c_{rent} + RoomScore \cdot c_{roomsize} + UniversityScore \cdot c_{universitydistance} + StationScore \cdot c_{stationdistance} + SupermarketScore \cdot c_{supermarketdistance} + AgeScore \cdot c_{buildingage})$$

$$c = weight$$



Fitness Function

```
[ ]  def fitness(apartment):
         return (
             apartment["RentScore"] * weights["Rent"]
             +
             apartment["RoomScore"] * weights["Room Size"]
             +
             apartment["UniversityScore"] * weights["University Distance"]
             +
             apartment["StationScore"] * weights["Station Distance"]
             +
             apartment["SupermarketScore"] * weights["Supermarket Distance"]
             +
             apartment["AgeScore"] * weights["Building Age"]
         )
```

The evaluation function is as of the following:

Initial Population

```
population_size = 30

population = filtered_df.sample(
    n=min(population_size, len(filtered_df)),
    replace=True
).index.tolist()
```

Evaluate Population

```
fitness_scores = []

for idx in population:

    score = fitness(filtered_df.loc[idx])

    fitness_scores.append(score)
```

- We then used Roulette Selection. Apartments with higher fitness are more likely to be selected.

Selection

```
import random

def roulette_selection(population, scores):

    total = sum(scores)

    probabilities = [s / total for s in scores]

    return random.choices(
        population,
        weights=probabilities,
        k=2
    )
```

For the crossover, we use

- One chromosome represents one apartment.
- Traditional one-point and two-point crossover are not suitable.
- One of the selected parents is randomly chosen as the child.

Crossover

```
def crossover(parent1, parent2):

    return random.choice([parent1, parent2])
```

Mutation:

- 10% chance of replacing the apartment with another random apartment.
- Helps maintain diversity in the population.

Mutation

```
def mutation(chromosome):
    if random.random() < 0.1:
        chromosome = random.choice(filtered_df.index.tolist())
    return chromosome
```

Then the evolution function is as of the following

Evolution

```
generations = 100
for g in range(generations):
    new_population = []
    while len(new_population) < population_size:
        p1, p2 = roulette_selection(population, fitness_scores)
        child = crossover(p1, p2)
        child = mutation(child)
        new_population.append(child)
    population = new_population
    fitness_scores = [
        fitness(filtered_df.loc[idx])
        for idx in population
    ]
```

In this fitness based selection, the children tend to replace the least fit individuals in the population. The selection of the least fit individuals may be done using a variation of any of the selection policies described before tournament selection, fitness proportionate selection, etc.

The code to print the result is the following

RESULT

```
filtered_df["Fitness"] = filtered_df.apply(fitness, axis=1)

ranked = filtered_df.sort_values(
    by="Fitness",
    ascending=False
)

result = ranked[
    [
        "Apartment name",
        "Rent (¥)",
        "Room size(m²)",
        "Floor plan",
        "Distance to University of Tsukuba (km)",
        "Distance to Tsukuba Station (km)",
        "Address",
        "Product URL",
        "Fitness"
    ]
].copy()

result["Fitness"] = result["Fitness"].round(3)

result.insert(0, "Rank", range(1, len(result) + 1))

display(result)
```

The result is the following. We obtained ranking system based on the user's preferred rating .

	Rank	Apartment name	Rent (¥)	Room size(m²)	Floor plan	Distance to University of Tsukuba (km)	Distance to Tsukuba Station (km)	Address	Product
70	1	賃貸アパート	37000	26.41	1K	2.10	1.6	茨城県つくば市春日３	https://suumo.jp/chintai/jnc_000108617359/?bc
12	2	ハイツ路川Ｉ	28000	20.00	1K	1.10	2.9	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000105687002/?bc
19	3	コートダジュールC	39000	26.40	1K	1.10	2.9	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000108715191/?bc
3	4	グリーンハイツ春日B	30000	20.46	1K	1.10	2.3	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000057560579/?bc
59	5	リバティマンション	30000	21.45	1K	2.50	1.8	茨城県つくば市天久保２	https://suumo.jp/chintai/jnc_000087091733/?bc
53	6	ワンダーロジュマン	30000	23.01	1K	2.10	1.6	茨城県つくば市天久保１	https://suumo.jp/chintai/jnc_000108476678/?bc
52	7	アーバンハイツ	35000	24.00	ワンルーム	1.10	2.9	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000050654375/?bc
15	8	沼尻ハイツA	25000	20.00	1K	1.10	2.9	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000082430281/?bc
65	9	ロマネスクつくば(Aタイプ)	28000	24.09	1K	4.60	2.0	茨城県つくば市松代２	https://suumo.jp/chintai/jnc_000076014882/?bc
64	10	ロマネスクつくば(Aタイプ)	28000	24.09	1K	4.60	2.0	茨城県つくば市松代２	https://suumo.jp/chintai/jnc_000080032171/?bc
32	11	賃貸アパート	30000	20.29	1K	1.10	2.9	茨城県つくば市春日４	https://suumo.jp/chintai/jnc_000018405149/?bc
5	12	サンテ・サクラ	40000	23.20	1K	0.85	4.1	茨城県つくば市桜２	https://suumo.jp/chintai/jnc_000092287086/?bc
10	13	コーポクラモチ	38000	22.00	1K	1.00	2.9	茨城県つくば市天久保４	https://suumo.jp/chintai/jnc_000108332302/?bc
60	14	賃貸マンション	30000	20.00	1K	1.90	2.7	茨城県つくば市天久保３	https://suumo.jp/chintai/jnc_000108338821/?bc
58	15	ディアスさくらC	40000	24.71	1K	1.70	4.4	茨城県つくば市桜３	https://suumo.jp/chintai/jnc_000108455500/?bc
44	16	賃貸マンション	30000	24.80	1K	5.30	2.6	茨城県つくば市松代４	https://suumo.jp/chintai/jnc_000108764487/?bc
43	17	賃貸マンション	35000	20.22	1K	2.50	1.8	茨城県つくば市天久保２	https://suumo.jp/chintai/jnc_000108198226/?bc